

Continuous action space:
from zero to hero

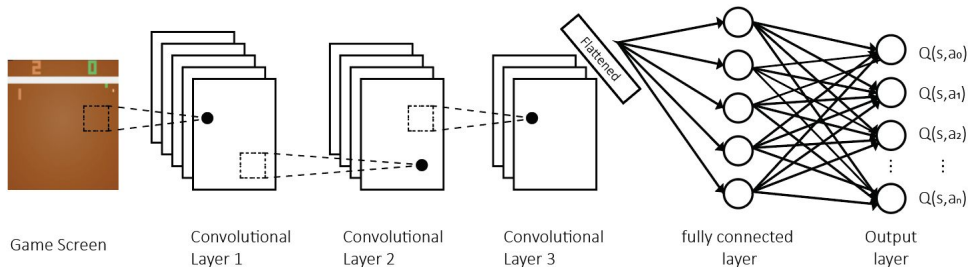
Where we left last time

Last time we saw 2 types of algorithms that work like a charm in the finite action space setting:

DQN

We create a network that for each state gives us all the Q estimate for each available action, and based on that, we handcraft a policy to map from Q values to probabilities (ϵ -greedy policy)

$$Q(s) : s \in \mathbb{R}^n \rightarrow \forall a, Q(s, a) \in \mathbb{R}$$



A2C/TRPO/PPO

Here, in addition to the Q||V network, we create another network that for each state gives us a distribution over action for that specific state, thus the policy is learnt

$$\pi(s) : s \in \mathbb{R}^n \rightarrow \forall a, p(a|s) \in [0, 1]$$

$$s.t. \sum_a p(a|s) = 1$$

What's the problem then?

What's the problem then in continuous actions (wrt DQN)

Well, in DQN, we had an output for each possible action, and this is no more possible, as you need an infinite number of output neurons.

Thus, we have to go back to the case of having a network that takes the action-space pair as input $Q(s,a)$, however now, we have a problem of “**which action do we select**” (or how to parameterize the policy)

One possible option is to parameterize the policy as follows, thus to **take an action proportionally to how “good” it is**:

$$\pi(a|s) \propto \exp\left(\frac{Q(s,a)}{T}\right)$$

What's the problem then in continuous actions (wrt DQN)

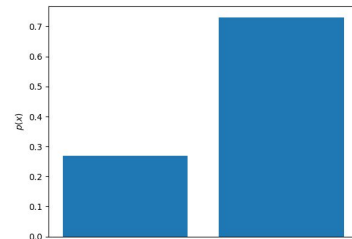
This is known as a **softmax/Boltzmann policy**:

$$\pi(a|s) \propto \exp\left(\frac{Q(s, a)}{T}\right)$$

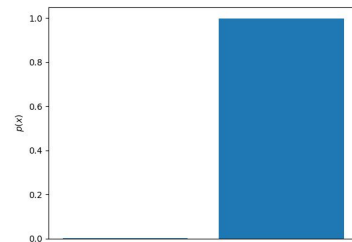
However, clearly there are 2 problems:

1. **What is a good T** (temperature)? Indeed, softmax is not invariant to the scale of the logits, thus we need to change T to slowly approach a greedy policy, however there is no good guess on how

softmax($v = [1, 2]$)



softmax($v = [10, 20]$)



What's the problem then in continuous actions (wrt DQN)

This is known as a **softmax/Boltzmann policy**:

$$\pi(a|s) \propto \exp\left(\frac{Q(s,a)}{T}\right)$$

However, clearly there are 2 problems:

1. What is a good **T** (temperature)?
2. That symbol is a “proportional to”... however, **how do we actually go from that to a policy/distribution?** we need to normalize, however the function is definitely not integrable wrt **a**, thus we have to use **Monte Carlo**

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\int \exp\left(\frac{Q(s,a)}{T}\right) da}$$

Is this the same Monte Carlo we know from RL?

Long story short... yes, but seen from another perspective. Monte Carlo tells us just that, given a probability distribution function $\mathbf{p}$ and a transformation of it $\mathbf{f}$:

$$\mathbb{E}[x] \approx \sum_{x \sim p(x)} x, \text{ but also } \mathbb{E}[f(x)] \approx \sum_{x \sim p(X)} f(x)$$

However, suppose to have any function $\mathbf{g}(\mathbf{x})$, that is not a PDF, and we want to integrate it, then we can reparametrize in the following way:

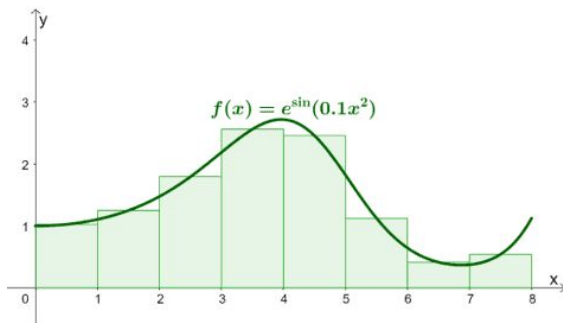
$$\int g(x)dx = \int \frac{p(x)}{p(x)}g(x)dx = \int \frac{g(x)}{p(x)}p(x)dx \approx \sum_{x \sim p(x)} \frac{g(x)}{p(x)}$$

Thus, we can **sample values from a known PDF $\mathbf{p}$, evaluate $\mathbf{g}$ on that point, and sum all the resulting values**, and we are guaranteed to converge (at some point)

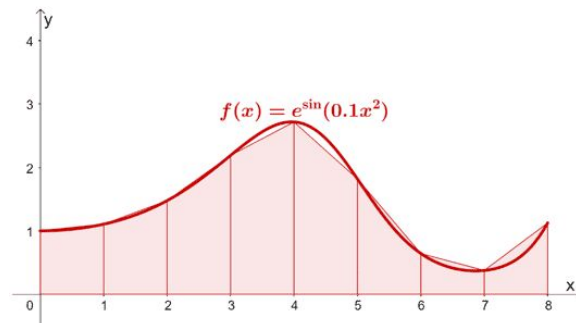
Do we even need all of this for a simple integration?

Well, not always, if you are in a low dimensional regime (bear in mind that in our case such space is the action space, thus for example the number of robotic joints), you can use another family of methods called **“Quadrature”**, that you definitely already know one instance of it... do you recall it?

Midpoint rule and trapezoids! (and if you have a really good memory, even Simpson's Rule)



Midpoint



Trapezoid

Are we done? Can we use it?

Now, since we can integrate any arbitrary function, we finally can approximate the denominator of our policy:

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\int \exp\left(\frac{Q(s,a)}{T}\right) da}$$

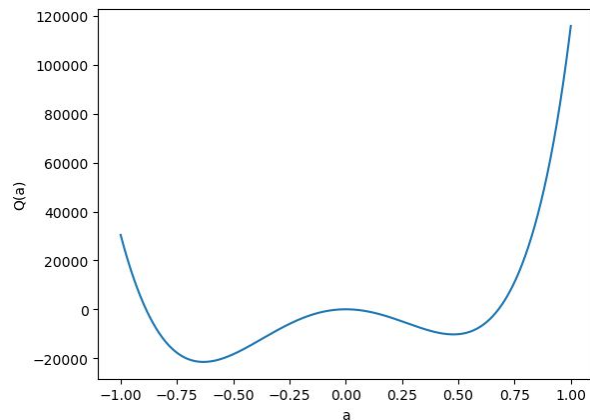
However, these family of integration methods work fine only in low dimensional space, in our case the space of the actions... and we are still left with the problem of picking the temperature **T**

What sort of policy is this even?

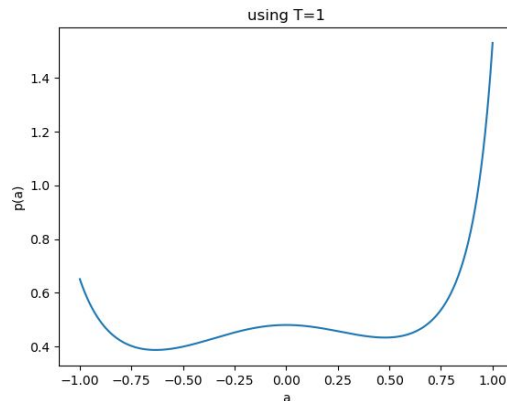
It's hard to imagine the type of policy that comes out of this monstrosity, thus I'll give you an example

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\int \exp\left(\frac{Q(s,a)}{T}\right) da}$$

Say we only have one state (so that the Q becomes $Q(a)$) and we only have 1 continuous action in $[-1, 1]$, with the following Q function:



Becomes

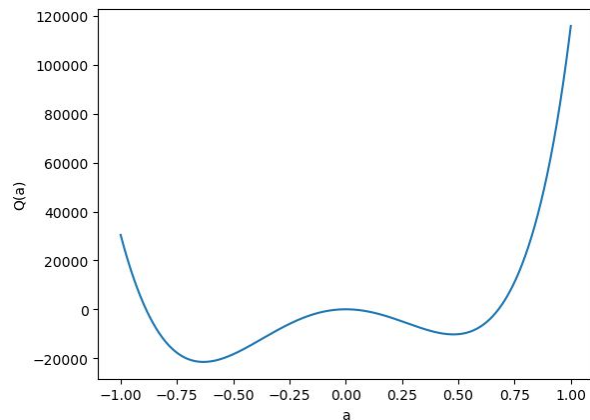


What sort of policy is this even?

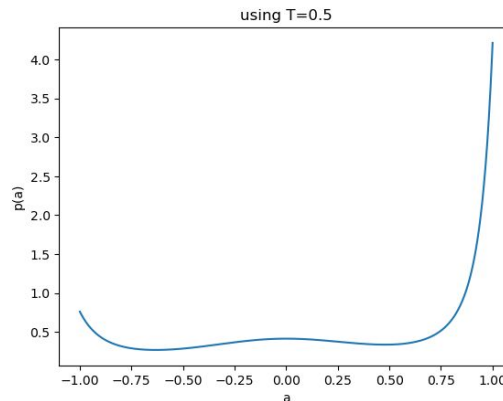
It's hard to imagine the type of policy that comes out of this monstrosity, thus I'll give you an example

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\int \exp\left(\frac{Q(s,a)}{T}\right) da}$$

Say we only have one state (so that the Q becomes $Q(a)$) and we only have 1 continuous action in $[-1, 1]$, with the following Q function:



Becomes

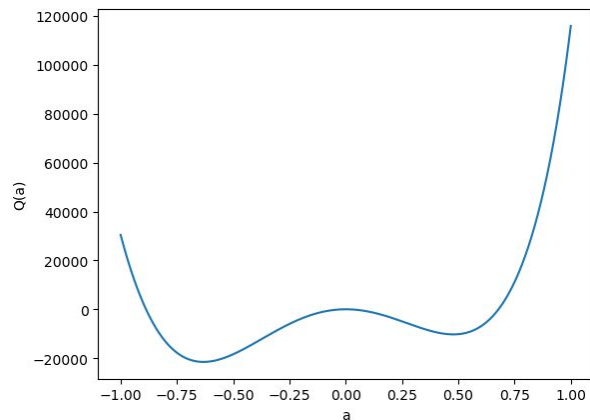


What sort of policy is this even?

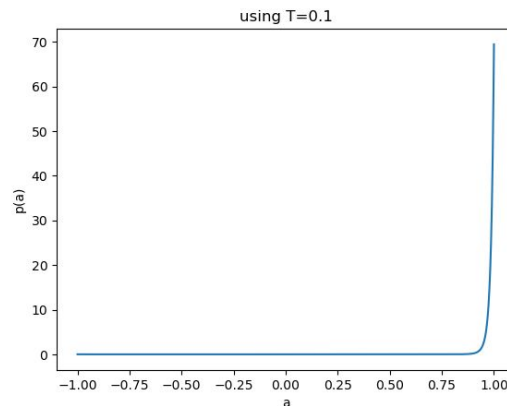
It's hard to imagine the type of policy that comes out of this monstrosity, thus I'll give you an example

$$\pi(a|s) = \frac{\exp\left(\frac{Q(s,a)}{T}\right)}{\int \exp\left(\frac{Q(s,a)}{T}\right) da}$$

Say we only have one state (so that the Q becomes $Q(a)$) and we only have 1 continuous action in $[-1, 1]$, with the following Q function:



Becomes



And what about sampling?

Well, when it comes to sampling, luckily for us, we can instead do everything in closed form, and the trick is very simple (most libraries implements random sampling from continuous distribution in this way), and it's called **Inverse transform sampling**:

- you take a **distribution** that **you know how to sample** (usually $U(0,1)$)
- you **sample a point** from such distribution
- you **calculate the CDF** (which for the uniform correspond to the sample)
- you find which value of “x” your “custom” distribution would have led to that CDF

This obviously implies that you have to know the integral of your “custom” distribution, but at this point you should know how to do that

(this until you have only 1 dimensions, then you have to make assumptions on then independence of actions, and using marginals as 1D variables)

And what about sampling?

Let's do an example using the uniform distribution, to sample from a Gaussian Distribution:

- **Sample a point from the uniform, 0.7** which CDF is $P(U(0, 1) < 0.7) = 0.7$
- Now, we take the Gaussian CDF and we ask our-self, ***“for which x do we get a CDF of 0.7”***: $P(N(0, 1) < x) = 0.7 \rightarrow F_{N(0,1)}(x) = 0.7$
- such x is **0.5244**, and to do that, we only need to use the inverse of the CDF of a Gaussian, which you can manually calculate

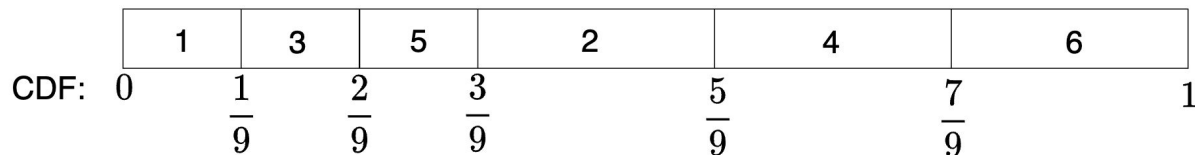
$$F_{N(0,1)}(x) = 0.7 \rightarrow x = F_{N(0,1)}^{-1}(0.7)$$

Now, to me this still seems a bit too abstract, however the same holds for discrete variables, and it's more visible this trick

And what about sampling?

Say we have an unfair dice, where odd numbers are sampled with $1/9$ probability, and even number with $2/9$ probability.

Since the order is not relevant, we can see this distribution as follows:



Say we sample a value from a **U(0,1)**, for example again 0.7; well, **5/9 < 0.7 < 7/9**, thus the discrete sample will be **4**... do you see why this work?

Because what's the probability that the random sample from the uniform distribution would have fallen in the range corresponding to 4?

$$F_{U(0,1)}\left(\frac{7}{9}\right) - F_{U(0,1)}\left(\frac{5}{9}\right) = \frac{7}{9} - \frac{5}{9} = \frac{2}{9}$$

What's the problem then in continuous actions (wrt PPO)

In advantage-actor-critic settings, **on the critic side, we don't have many problems**, as the value definition already works in continuous settings:

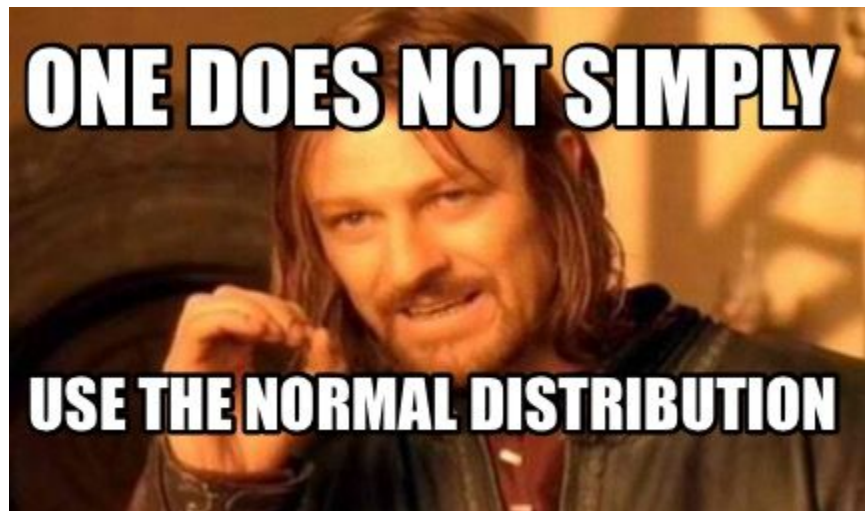
$$V^{\pi}(s) = \int_C Q(s, a)p(s, a)da, a \in C$$

However, as per earlier, **we just moved the problem on the policy side**, and that's even a bigger problem, because **now we have to define a distribution over infinite points, that also depends on the state**.

One possible option is to discretize the action space and it's been done in some cases, the most famous being RT-2 (evolution of PaLM-E, Google), however it's very empirical this solution, and doesn't always work...

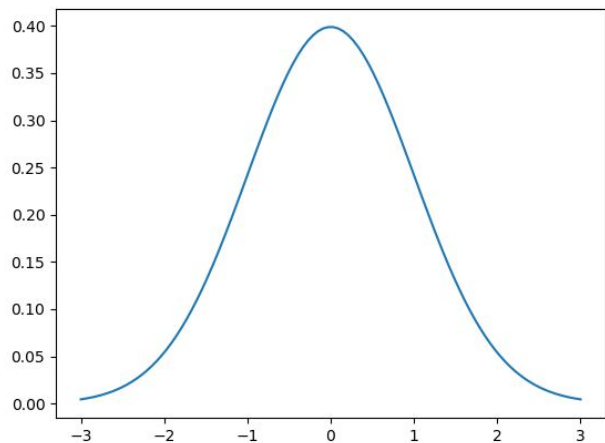
Constraining to a family of distributions

One option is to limit us to model the policy with a certain **family of parameterized distributions** that we can easily manipulate... which one will we use?



Constraining to a family of distributions

One option is to limit us to model the policy with a certain **family of parameterized distributions** that we can easily manipulate... which one will we use?



Indeed, what we might assume is that our policy might be locally Gaussian, and the nice part of limiting us to a family of **parametric** distributions is that... it's parametric!

Thus the whole PDF is characterized by a finite number of parameters.

Thus our policy just needs to **predict the parameters of the distribution**, and we will use the magic of RL to improve only the parameters

One example is worth a thousand words

Let's step back from the world of DL for a second, and imagine we have the following problem:

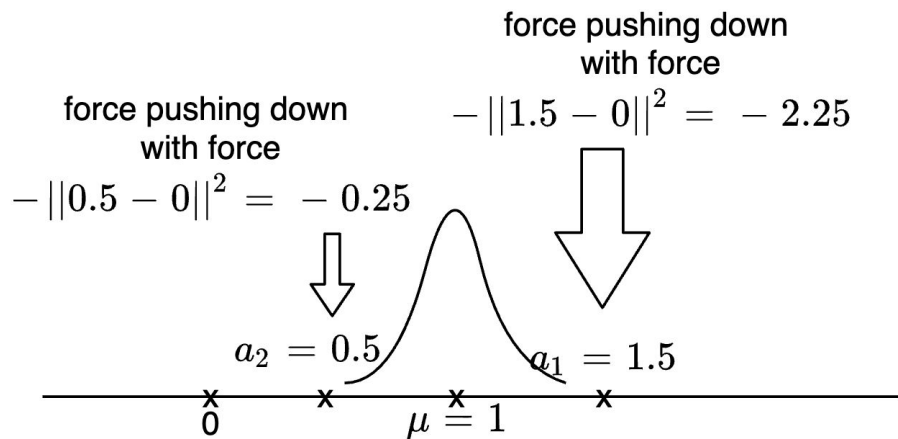
- the state is only one, so we don't need to model it
- the action is 1-dimensional and it is continuous in $\mathbb{R}$
- the reward is the following $-\|\mathbf{a} - \mathbf{origin}\|^2$, thus the best action is the 0-action

Say we parameterize the policy with a Gaussian with fixed variance, thus we only want to learn μ

What we can do, is to sample from $\mathbf{N}(\mu, 1)$, and use RL to optimize μ

One example is worth a thousand words

To do so, say we initialize $\mu=1$, and we sample 2 values from the distribution, **1.5** and **0.5**:



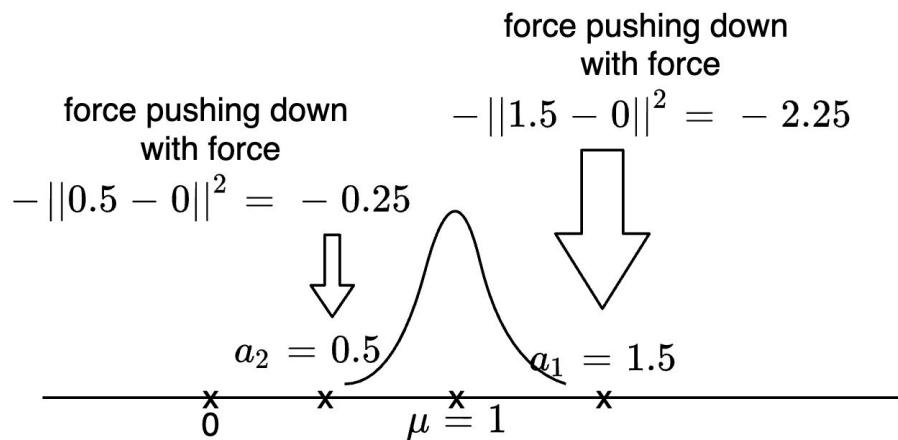
Given the 2 samples, we need to update μ using Gradient Ascent and REINFORCE:

$$\begin{aligned}\nabla_{\mu} L &= R \cdot \nabla_{\mu} \ln p(a|\mu) \\ &\approx R \cdot \ln e^{-(a-\mu)^2} \\ &= R \cdot -(a - \mu)^2\end{aligned}$$

$$\mu' = \mu + \alpha \nabla_{\mu} L$$

One example is worth a thousand words

To do so, say we initialize $\mu=1$, and we sample 2 values from the distribution, **1.5** and **0.5**:



Now, the gradient of the actions, since the distribution is symmetric, are the same in these 2 cases, thus the only term that “controls” how to update μ are the rewards.

As you can see, μ will be “pushed away” from 1.5 harder than from 0.5, thus, since it can’t be pushed away from both parts, it will be pushed away only from the strongest, 1.5, going towards 0.

Gaussian PPO

Now, the last step is to just parametrize $\mu = \pi(s)$, thus allowing for different states to have different distributions (just like in the discrete case), in other words, we just need to fix the PPO algorithm a bit:

1. given s , calculate $\mu = \pi(s)$, sample an action $a \sim N(\mu, 1)$
2. act on the environment and observe r, s'
3. optimize the V function using TD learning (just like we used to do in discrete settings)
4. optimize the loss, using PPO: now note that ppo requires to calculate the importance sampling ratio, but we can easily do that using Gaussians, and same holds for the KL-regularized version, as the KL between 2 Gaussians can be calculated in closed form (try at home if you have experience with Calc 1/2)

THATS ALL YOU GOT? REALLY ?





THATS ALL YOU GOT? REALLY ?

BUT WAIT..

**WELL, WE CAN DO BETTER THAN
THAT.**

What are we even doooooiiing?

Let's go back to the example earlier, and think what we are doing. **We are perturbing our continuous action a bit**, to understand if increasing it or decreasing it will improve the policy... can we do better than actually empirically testing **if that new action is an improvement?** Or do we already have something that tells us that?...

$$Q^*(s,a)$$

At the end of the day, we already have a tool that tells us how that perturbation is going to work, thus we can just rely on that estimate... or do we?

Well, for this to work, we need the exact $Q^*(s,a)$ under that policy, and it's not clear how to get it... but **maybe we just need an approximation**

Is perturbation even necessary?

We just saw that if we **locally perturb an action**, we can use $Q(s,a)$ to see if that's an improvement or not... does this seem familiar to you? (something seen during Calc 1)

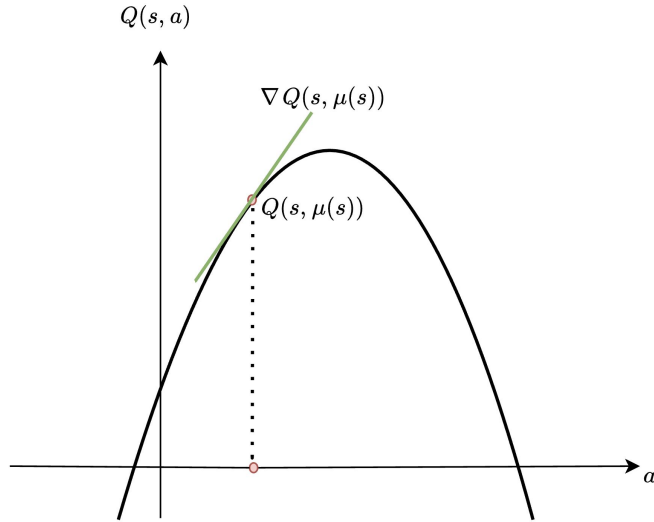
Hint 1: we want to check if $Q(s, a) > Q(s, a + \epsilon)$ and in case, **move in that direction**... is there a mathematical operator that tells us this property?

Hint 2: let me rephrase the above statement moving everything on the left, and taking a limit:

$$\lim_{\epsilon \rightarrow 0} \frac{Q(s, a + \epsilon) - Q(s, a)}{\epsilon}$$

Derivatives have always been your best friend

If we have a **differentiable model** of $Q(s,a)$, why not use that as for the improvement?
And obviously, a great choice for such model, is a **neural network**.



Say that this parabola is the function that the network has learnt as $Q(s,a)$ for a certain state s , **we can take the derivative of such function wrt the action** (green bar), and then using the chain rule, take the derivative of that action wrt the policy network

$$\frac{dQ(s, a)}{da} \frac{da}{d\theta} = \frac{dQ(s, \mu_{\theta}(s))}{d\mu_{\theta}(s)} \frac{d\mu_{\theta}(s)}{d\theta}$$

Welcome, DDPG

DDPG (**D**eep **D**eterministic **P**olicy **G**radient) is indeed an algorithm that does “just” that:

- **Deep**: uses deep neural network to model both policy and q-network
- **Deterministic**: we don't need anymore the REINFORCE operator that estimates the gradient
- **Policy Gradient**: as it improves a policy... but what policy is it even improving?

Welcome, DDPG

DDPG (**D**eep **D**eterministic **P**olicy **G**radient) is indeed an algorithm that does “just” that:

- **Deep**: uses deep neural network to model both policy and q-network
- **Deterministic**: we don't need anymore the REINFORCE operator that estimates the gradient
- **Policy Gradient**: as it improves a policy... but what policy is it even improving?

Well... the greedy one... in fact, we can see the policy as following:

$$\mu(s) = \operatorname{argmax}_a Q(s, a)$$

What about exploration?

Welcome, DDPG

Given that we can see the learnt policy as follows:

$$\mu(s) = \operatorname{argmax}_a Q(s, a)$$

we can reformulate the **DQN algorithm in the following way** (since our policy will already tell us the best action):

$$y = r + \gamma \max_a Q(s, a) \Rightarrow y = r + \gamma Q(s, \mu(s))$$

And thanks to this re-parametrization, we are back at **Q-learning**, and this this algorithm is **completely off-policy**... in other words, the fact that we learn the optimal/greedy one, it's not a problem, because we can **use a behavioural policy during training**, and use that experience to optimize the greedy one

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

Pretty much just
DQN

Actual DDPG

Stabilize gradient,
used also in “plain”
DQN

Issues with DDPG

If something is too good to be true, probably it is. DDPG carries with him a lot of problems:

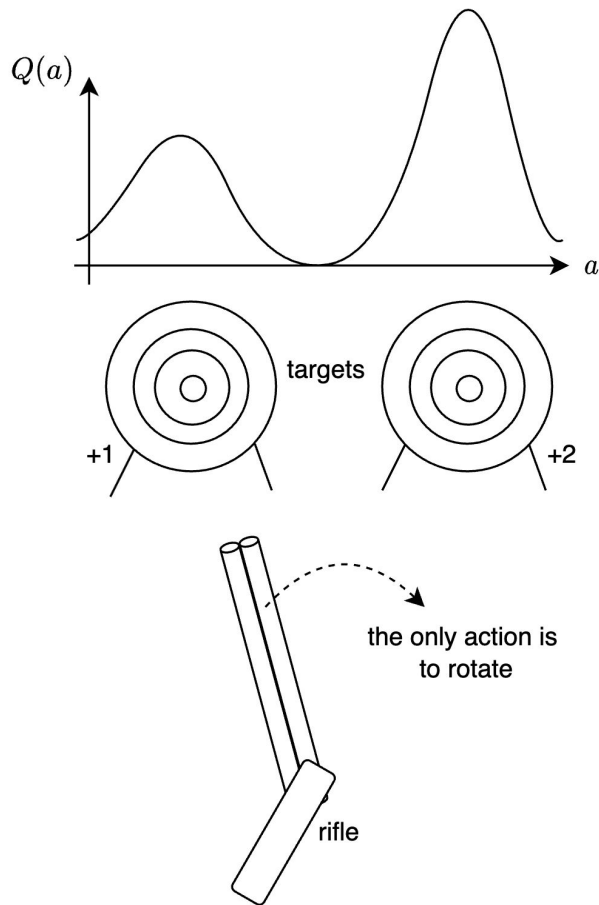
- **it needs a good model of $Q(s,a)$** so that the gradient with respect to the action is something correlated with the true gradient, where instead in REINFORCE we use the environment to actually estimate that gradient
- **overestimation**: it's not such a big problem with plain DQN, as the only important part is the order of the Q of the actions, however here the overestimation causes bigger gradient, making the learning unstable
- **non-convexity of $Q(s,a)$ with respect to the action**: why is the non convexity of the Q a problem?

Issues with DDPG

First, let's clarify that the non convexity that we are talking about, has nothing to do with the non-convexity of neural networks. Indeed, we might estimate the exact $Q^*(s,a)$ using simulations, taking the NN out of the equation. However, we might have a **non convexity with respect to the action**, thus being intrinsically a problem of the environment.

Issues with DDPG

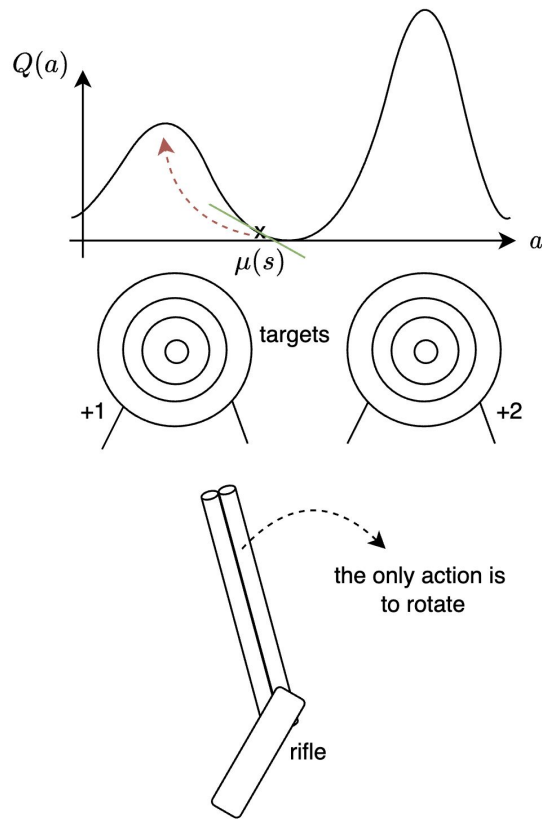
In this example, if we start with an initial guess being more **to the left than the middle point**, the **left mode** with max 1 points **will be an attractor** if we follow the gradient.



Issues with DDPG

In this example, if we start with an initial guess being more **to the left than the middle point**, the **left mode** with max 1 points **will be an attractor** if we follow the gradient.

The worst part, is that this happens **even having a perfect model of the word** (having $Q^*(s,a)$), where instead with discrete actions we can jump from one mode to the other with no problems... this means that **even though we might have a behavioural policy that explores both the modes**, the greedy one **might never converge to the optimal**.



Improvements of DDPG:

Even though we have no chances to solve this problem, we can still tackle the overestimation problem, and the stability problem.

Two algorithms that aim to improve DDPG are:

- **TD3: Twin Delayed Deep Deterministic Policy Gradient**, which adds a second network, and uses the **minimum between the two as a target for the Q-learning**, and adds noise also to the Q-learning target action (smoothing)
- **SAC: Soft Actor Critic**, which adds in addition to what TD3 proposes, an entropy bonus to the learning objective, thus not only modelling μ , but also σ of the policy, and using the “**Reparametrization Trick**” to train both of them without using Reinforcement Learning

Maximum Entropy RL and Soft Actor Critic

Soft Actor Critic is mostly a DDPG with an **entropy bonus**, and the two targets as TD3. However, the entropy can be calculated for a distribution, however we have none in DDPG.

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))] \quad J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi_\phi(\cdot | \mathbf{s}_t) \parallel \frac{\exp(Q_\theta(\mathbf{s}_t, \cdot))}{Z_\theta(\mathbf{s}_t)} \right) \right]$$

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} [\log \pi_\phi(f_\phi(\epsilon_t; \mathbf{s}_t) | \mathbf{s}_t) - Q_\theta(\mathbf{s}_t, f_\phi(\epsilon_t; \mathbf{s}_t))]$$

To do so, **SAC** employs a trick, called “reparameterization trick” to also estimate the variance of an action. Now, every action instead of being a single value, is parametrized by a Gaussian policy $\mathbf{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$... However, how do we learn this additional parameter?

$$D_{KL}(p||q) = \int_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)} dx \quad \mathcal{H}(p) = \int_{x \in \mathcal{X}} p(x) \log p(x) dx$$

The reparameterization trick

We cannot just pass both μ and σ to the Q function, because Q takes a state and an action, and we not even can use REINFORCE to learn it, otherwise we lose every advantage of DDPG.

Instead, they use a reparametrization trick, this works like a charm, and it exploits the following fact of Gaussian distribution:

$$N(\mu, \sigma^2) \rightarrow \mu + N(0, \sigma^2) \rightarrow \mu + N(0, 1) \cdot \sigma$$

In other words, now we can just do the following:

$$\pi(s) \rightarrow [\vec{\mu}, \vec{\sigma}], \epsilon \sim N(0, 1) \Rightarrow a = \vec{\mu} + \vec{\sigma} \cdot \epsilon$$

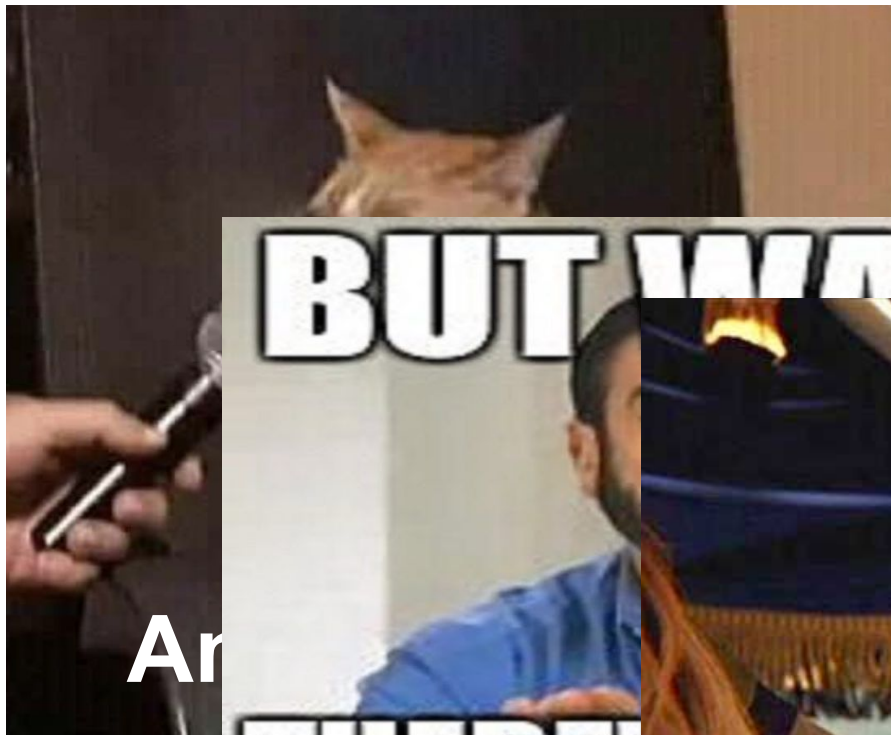


Anything else



Anything





Evolutionary methods

If RL feels like trial and errors to you, wait to see what Evolutionary Methods will do...

EM are a wide range of methods that borrows from evolution the idea of “**only the best survives**”, some keywords that you can lookup on google are:

- Genetic algorithms
- Evolutionary strategies
- Genetic programming
- Evolutionary programming

We will briefly present 2/3 of them

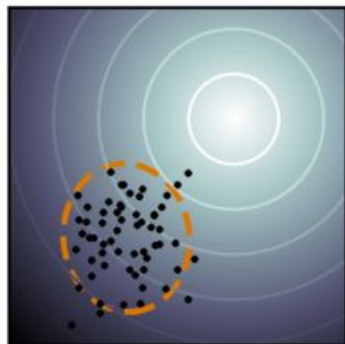
Cross Entropy Method (CEM)

It's literally **RL in the weight space**:

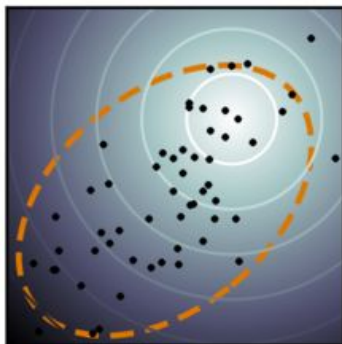
```
Initialize  $\mu \in \mathbb{R}^d, \sigma \in \mathbb{R}^d$   
for iteration = 1, 2, ... do  
    Collect n samples of  $\theta_i \sim N(\mu, \text{diag}(\sigma))$   
    Perform a noisy evaluation  $R_i \sim \theta_i$   
    Select the top  $p\%$  of samples (e.g.  $p = 20$ ), which we'll  
        call the elite set  
    Fit a Gaussian distribution, with diagonal covariance,  
        to the elite set, obtaining a new  $\mu, \sigma$ .  
end for  
Return the final  $\mu$ .
```

You can see this as “Policy Gradient” in the weight space, as **you reinforce the weight proportionally to their return in the simulations**

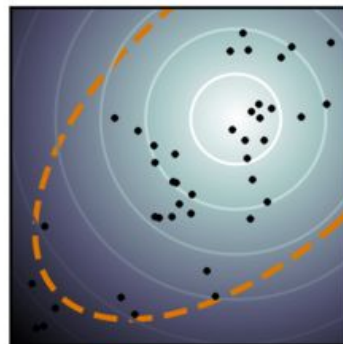
Generation 1



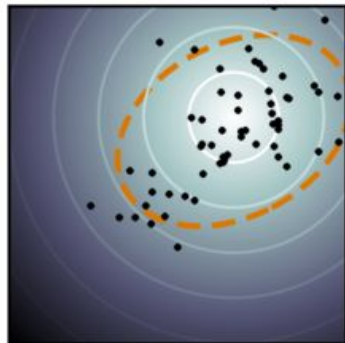
Generation 2



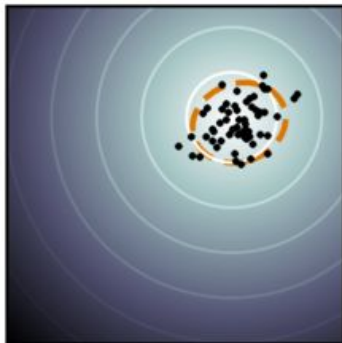
Generation 3



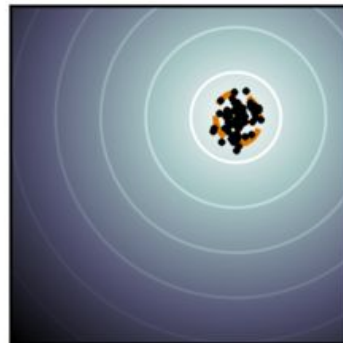
Generation 4



Generation 5



Generation 6



Covariance Matrix Adaptation – Evolutionary Strategy (CMA-ES)

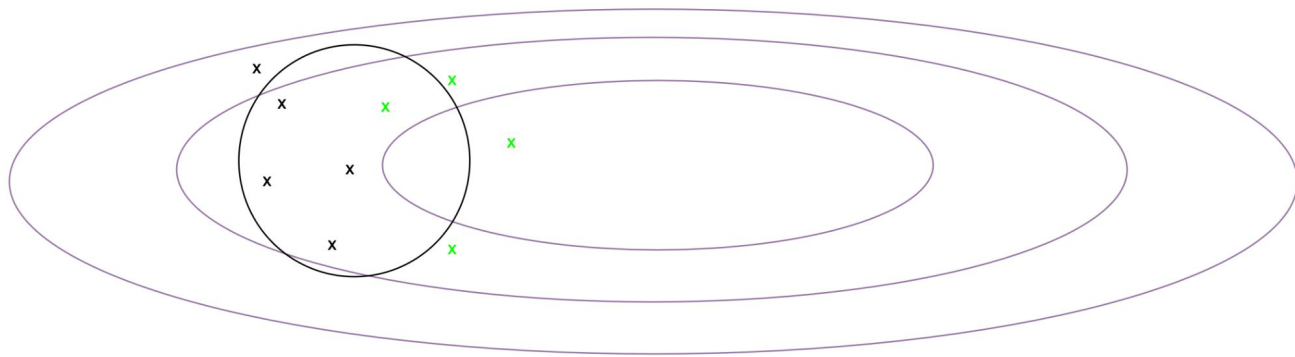
CMA-ES is the new version of CEM, where instead of fitting a diagonal N-dimensional Gaussian distribution at each step, **you slowly adapt its covariance matrix using a rank-K matrix that you get using only the top-K samples**, and a **rank-1 matrix that is an exponential moving average of the previous step** (pretty much a momentum term)

The nice part is that this type of methods usually converge to different minima, thus the policy that you get out of them are much more diverse than the ones that Policy-gradient methods produce, mainly due to their random-nature.

Covariance Matrix Adaptation – Evolutionary Strategy (CMA-ES)

The algorithm pretty much starts by having a μ vector of initial mean values, σ vector of initial std values (stepsize), and a covariance matrix $\mathbf{C}$ initialized as the identity.

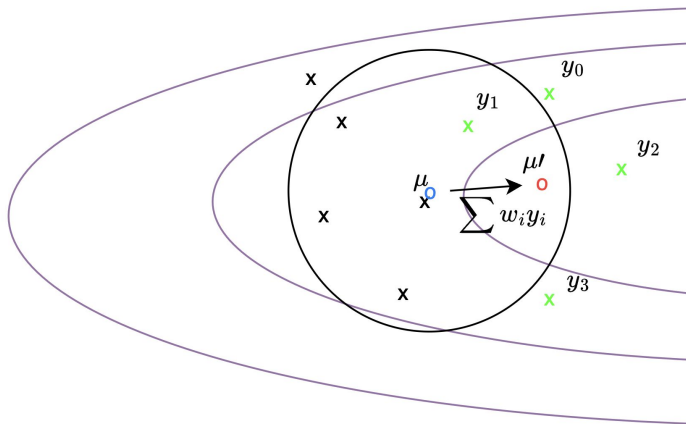
1. sample N points, evaluate them and take the top K



Covariance Matrix Adaptation – Evolutionary Strategy (CMA-ES)

The algorithm pretty much starts by having a μ vector of initial mean values, σ vector of initial std values (step-size), and a covariance matrix $\mathbf{C}$ initialized as the identity.

1. sample N points, evaluate them and take the top K
2. find new center and update it: $\mu' = \mu + \sigma \sum_{i=0}^K w_i y_i$



Covariance Matrix Adaptation – Evolutionary Strategy (CMA-ES)

The algorithm pretty much starts by having a $\boldsymbol{\mu}$ vector of initial mean values, $\boldsymbol{\sigma}$ vector of initial std values (step-size), and a covariance matrix $\mathbf{C}$ initialized as the identity.

1. sample N points, evaluate them and take the top K
2. find new center and update it
3. update the momentum for the covariance matrix

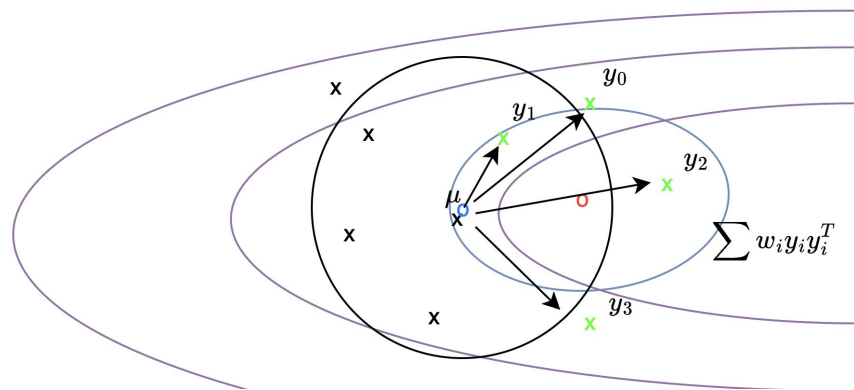
$$\mathbf{p}_c^{(g+1)} = (1 - c_c)\mathbf{p}_c^{(g)} + \sqrt{c_c(2 - c_c)}\mu_{\text{eff}} \frac{\mathbf{m}^{(g+1)} - \mathbf{m}^{(g)}}{c_m\sigma^{(g)}}$$

Covariance Matrix Adaptation – Evolutionary Strategy (CMA-ES)

The algorithm pretty much starts by having a μ vector of initial mean values, σ vector of initial std values (step-size), and a covariance matrix C initialized as the identity.

1. sample N points, evaluate them and take the top K
2. find new center and update it
3. update the momentum for the covariance matrix
4. update the covariance matrix (yy^T is rank K)

$$C' = \beta(\alpha C + (1 - \alpha) \sum w_i y_i y_i^T) + (1 - \beta) p p^T$$



(Separable) Natural Evolutionary Strategy

From the original paper <https://arxiv.org/pdf/1106.4487> (Canonical Search Gradient algorithm):

If we use θ to denote the parameters of distribution $\pi(\mathbf{z} | \theta)$ and $f(x)$ to denote the fitness function for samples $\mathbf{z}$, we can write the expected fitness under the search distribution as

$$J(\theta) = \mathbb{E}_{\theta}[f(\mathbf{z})] = \int f(\mathbf{z}) \pi(\mathbf{z} | \theta) d\mathbf{z}. \quad (1)$$

The so-called ‘log-likelihood trick’ enables us to write

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int f(\mathbf{z}) \pi(\mathbf{z} | \theta) d\mathbf{z} \\ &= \int f(\mathbf{z}) \nabla_{\theta} \pi(\mathbf{z} | \theta) d\mathbf{z} \\ &= \int f(\mathbf{z}) \nabla_{\theta} \pi(\mathbf{z} | \theta) \frac{\pi(\mathbf{z} | \theta)}{\pi(\mathbf{z} | \theta)} d\mathbf{z} \\ &= \int \left[f(\mathbf{z}) \nabla_{\theta} \log \pi(\mathbf{z} | \theta) \right] \pi(\mathbf{z} | \theta) d\mathbf{z} \\ &= \mathbb{E}_{\theta} [f(\mathbf{z}) \nabla_{\theta} \log \pi(\mathbf{z} | \theta)]. \end{aligned}$$

(Separable) Natural Evolutionary Strategy

However, such strategy does not quite never converge to the optima, but zig-zags around it (see original paper). Instead of using the exact log-trick, they use a different one:

$$\begin{aligned} \max_{\delta\theta} J(\theta + \delta\theta) &\approx J(\theta) + \delta\theta^\top \nabla_\theta J, \\ s.t. \ D(\theta + \delta\theta || \theta) &= \varepsilon, \end{aligned}$$

Looks familiar?

And if so, given such similarity, what do you gain?

(Separable) Natural Evolutionary Strategy

However, such strategy does not quite never converge to the optima, but zig-zags around it (see original paper). Instead of using the exact log-trick, they use a different one:

Algorithm 3: Canonical Natural Evolution Strategies

input: f, θ_{init}

repeat

for $k = 1 \dots \lambda$ **do**

 draw sample $\mathbf{z}_k \sim \pi(\cdot|\theta)$

 evaluate the fitness $f(\mathbf{z}_k)$

 calculate log-derivatives $\nabla_{\theta} \log \pi(\mathbf{z}_k|\theta)$

end

$\nabla_{\theta} J \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} \nabla_{\theta} \log \pi(\mathbf{z}_k|\theta) \cdot f(\mathbf{z}_k)$

$\mathbf{F} \leftarrow \frac{1}{\lambda} \sum_{k=1}^{\lambda} \nabla_{\theta} \log \pi(\mathbf{z}_k|\theta) \nabla_{\theta} \log \pi(\mathbf{z}_k|\theta)^{\top}$

$\theta \leftarrow \theta + \eta \cdot \mathbf{F}^{-1} \nabla_{\theta} J$

until *stopping criterion is met*

(Separable) Natural Evolutionary Strategy

They further go and propose multiple adaptation for diverse settings, but the final SNES algorithm is remarkably straightforward:

Algorithm 7: Separable NES (SNES)

input: f , $\boldsymbol{\mu}_{init}$, $\boldsymbol{\sigma}_{init}$

repeat

for $k = 1 \dots \lambda$ **do**

 draw sample $\mathbf{s}_k \sim \mathcal{N}(0, \mathbb{I})$

$\mathbf{z}_k \leftarrow \boldsymbol{\mu} + \boldsymbol{\sigma} \mathbf{s}_k$

 evaluate the fitness $f(\mathbf{z}_k)$

end

 sort $\{(\mathbf{s}_k, \mathbf{z}_k)\}$ with respect to $f(\mathbf{z}_k)$ and compute utilities u_k

 compute gradients $\nabla_{\boldsymbol{\mu}} J \leftarrow \sum_{k=1}^{\lambda} u_k \cdot \mathbf{s}_k$
 $\nabla_{\boldsymbol{\sigma}} J \leftarrow \sum_{k=1}^{\lambda} u_k \cdot (\mathbf{s}_k^2 - 1)$

 update parameters $\boldsymbol{\mu} \leftarrow \boldsymbol{\mu} + \eta_{\boldsymbol{\mu}} \cdot \boldsymbol{\sigma} \cdot \nabla_{\boldsymbol{\mu}} J$
 $\boldsymbol{\sigma} \leftarrow \boldsymbol{\sigma} \cdot \exp(\eta_{\boldsymbol{\sigma}}/2 \cdot \nabla_{\boldsymbol{\sigma}} J)$

until *stopping criterion is met*;

Why would we want to use them?

Well now, you have no problem at all about the distribution of the policy of your network, as this method will just throw random actions, and keeps the candidate where the random actions led to the best solutions (you can see this also as an **approximation of finite-difference method with random direction, or as a directional derivative**)...

$$\lim_{h \in R^n \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

In addition to this, it's very **easy to parallelize/distribute**, since each worker can **sample one set of parameters according to a seed**, and then to the central orchestrator **just the seed it used** to generate the parameters, **and the return** it got!

Finally, please note that at no point of these two methods we used gradients (in fact they **are also known as derivative free/zero order methods**), thus **you can use models/neural networks that are not differentiable** (you might also have deduced it because we said this is Policy Gradient in the space settings... and Policy Gradient is “almost” just a gradient estimator...)

Congrats, you made to the end!
See you at the Labs, where we will implement some of this
algorithms!